

JPA Cascading, Lazy Loading and the N+1 Problem

JPA relationships do more than connect two entity classes. They also raise three practical questions:

1. What should happen to related entities when one entity is saved or deleted?
2. When should related data be loaded from the database?
3. How can we avoid executing a large number of hidden SQL queries?

These questions are handled through cascading, fetch strategies and query-level fetch plans.

1. Cascading

What Is a Cascade?

A cascade propagates a persistence operation from one entity to another through a mapped relationship.

Suppose an `Order` contains multiple `OrderItem` objects:

```
@Entity
@Table(name = "orders")
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToMany(
        mappedBy = "order",
        cascade = CascadeType.PERSIST
    )
```

```
private List<OrderItem> items = new ArrayList<>();  
}
```

```
@Entity  
public class OrderItem {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    @ManyToOne  
    @JoinColumn(name = "order_id")  
    private Order order;  
}
```

Now consider:

```
entityManager.persist(order);
```

Because the relationship uses `CascadeType.PERSIST`, Hibernate also persists the items contained in `order.getItems()`.

Conceptually, the single call behaves like this:

```
persist(order);  
persist(item1);  
persist(item2);  
persist(item3);
```

The application does not need to call `persist()` separately for every `OrderItem`.

Cascade Does Not Mean Relationship Ownership

Cascading and relationship ownership solve different problems:

- **Relationship ownership** decides which side controls the foreign-key value.

- **Cascading** decides whether a persistence operation should propagate to a related entity.

In the example above:

```
@ManyToOne
@JoinColumn(name = "order_id")
private Order order;
```

`OrderItem` is the owning side because it contains the `order_id` foreign key.

The `Order.items` collection is the inverse side because it uses:

```
mappedBy = "order"
```

Adding an item only to the inverse collection is not enough to set the foreign key correctly. Both sides of the object relationship should be maintained:

```
public void addItem(OrderItem item) {
    items.add(item);
    item.setOrder(this);
}
```

Usage:

```
Order order = new Order();

OrderItem item1 = new OrderItem();
OrderItem item2 = new OrderItem();

order.addItem(item1);
order.addItem(item2);

entityManager.persist(order);
```

The helper method keeps the Java object graph consistent, while `CascadeType.PERSIST` allows the items to be persisted through the order.

Cascade Is Not Enabled by Default

JPA does not automatically cascade every operation across relationships.

If no cascade is configured:

```
@OneToMany(mappedBy = "order")
private List<OrderItem> items;
```

then:

```
entityManager.persist(order);
```

does not automatically persist new `OrderItem` objects.

This is intentional. Related entities do not always share the same lifecycle.

2. `CascadeType.PERSIST`

`CascadeType.PERSIST` propagates the persist operation.

```
@OneToMany(
    mappedBy = "order",
    cascade = CascadeType.PERSIST
)
private List<OrderItem> items;
```

When a new order is persisted:

```
entityManager.persist(order);
```

new items reachable through the relationship are persisted as well.

This cascade is useful when:

- the parent and children are created together;
- the child belongs to the parent's lifecycle;

- saving the parent should also save its newly created children.

It should not be added merely for convenience. The correct question is:

If this parent is persisted, should these related objects always be persisted with it?

3. `CascadeType.REMOVE`

`CascadeType.REMOVE` propagates deletion from one entity to related entities.

```
@OneToMany(  
    mappedBy = "order",  
    cascade = CascadeType.REMOVE  
)  
private List<OrderItem> items;
```

Now:

```
entityManager.remove(order);
```

conceptually becomes:

```
remove(Order)  
├─ remove(Item A)  
├─ remove(Item B)  
└─ remove(Item C)
```

Possible SQL:

```
DELETE FROM order_item  
WHERE id = 1;  
  
DELETE FROM order_item  
WHERE id = 2;
```

```
DELETE FROM order_item
WHERE id = 3;
```

```
DELETE FROM orders
WHERE id = 100;
```

The child rows are normally deleted before the parent row so that the foreign-key constraint is not violated.

When Does **REMOVE** Make Sense?

`CascadeType.REMOVE` makes sense when the child belongs exclusively to the parent.

An `OrderItem` normally has no independent meaning without its `Order`. If the order is deleted, deleting its items is usually reasonable.

It can be dangerous when the related entity is shared.

For example:

```
@ManyToMany
private Set<Course> courses;
```

If many students share the same course, deleting one student must not delete the course itself. Therefore, `CascadeType.REMOVE` is usually inappropriate on shared many-to-many relationships.

4. Other Cascade Types

JPA provides the following cascade types:

```
CascadeType.PERSIST
CascadeType.MERGE
CascadeType.REMOVE
CascadeType.REFRESH
CascadeType.DETACH
CascadeType.ALL
```

Their meaning can be summarized as follows:

Cascade type	Propagated operation
<code>PERSIST</code>	Persist new related entities
<code>MERGE</code>	Merge the state of related detached entities
<code>REMOVE</code>	Remove related entities
<code>REFRESH</code>	Reload related entities from the database
<code>DETACH</code>	Detach related entities from the persistence context
<code>ALL</code>	Apply all the cascade types above

Multiple cascade types can be selected:

```
@OneToMany(
    mappedBy = "order",
    cascade = {
        CascadeType.PERSIST,
        CascadeType.REMOVE
    }
)
private List<OrderItem> items;
```

`CascadeType.ALL` should be used only when the related entity genuinely shares the complete lifecycle of the parent.

5. Relationship Loading

Fetching an entity by its primary key is straightforward:

```
Student student =
    entityManager.find(
        Student.class,
        1L
    );
```

However, a `Student` may be related to many other entities:

```

Student
├── Department
├── Address
├── Profile
├── Courses
├── Projects
├── Certificates
└── Comments

```

Those entities may contain further relationships of their own.

If Hibernate loaded every reachable entity whenever one `Student` was requested, a small operation could load a huge portion of the database.

Hibernate therefore needs a fetch plan:

Which relationships are required now, and which relationships can wait until they are actually used?

6. Static Fetch Strategy

A fetch strategy can be configured directly in the entity mapping:

```

@ManyToOne(fetch = FetchType.LAZY)
private Department department;

```

This is called a **static fetch strategy** because it becomes part of the entity's mapping metadata.

It acts as the default whenever the entity is loaded:

```

Student student =
    entityManager.find(
        Student.class,
        1L
    );

```


The default can still be overridden for a particular use case through a fetch join, an entity graph or another query-specific fetch plan.

7. `FetchType.LAZY` and `FetchType.EAGER`

JPA defines two fetch types:

```
FetchType.LAZY  
FetchType.EAGER
```

Conceptually:

```
LAZY  
Load the relationship when it is first needed.  
  
EAGER  
Load the relationship as part of loading the entity.
```

There is also an important specification-level distinction:

- `EAGER` is a requirement. The persistence provider must make the relationship available eagerly.
- `LAZY` is a hint. The provider is asked to delay loading, but the specification does not absolutely require it to do so.

Hibernate supports lazy loading extensively:

- to-many relationships are represented using lazy persistent collections;
- to-one relationships can be loaded lazily through proxies or bytecode enhancement.

8. Default Fetch Types

JPA defines the following defaults:

Relationship	Default fetch type
--------------	--------------------

@OneToMany	LAZY
@ManyToMany	LAZY
@ManyToOne	EAGER
@OneToOne	EAGER

Therefore, to-many relationships are lazy by default, while to-one relationships are eager by default.

These defaults should not automatically be treated as the best choice for every application. In practice, many teams explicitly configure to-one relationships as lazy:

```
@ManyToOne(fetch = FetchType.LAZY)
private Department department;
```

This keeps the default object graph small and allows each query to request the relationships required by its use case.

9. How Lazy Loading Works

Consider:

```
@Entity
public class Student {

    @Id
    private Long id;

    private String name;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id")
    private Department department;
}
```

When the student is loaded:

```
Student student =
    entityManager.find(
        Student.class,
        1L
    );
```

Hibernate can initially execute:

```
SELECT
    id,
    name,
    department_id
FROM student
WHERE id = 1;
```

The `Student` is loaded, but its `Department` may be represented by an uninitialized proxy.

Conceptually:

```
Student
├─ id = 1
├─ name = Rahul
└─ department = not initialized yet
```

Accessing only the department reference does not necessarily initialize it:

```
Department department =
    student.getDepartment();
```

Accessing data that requires the real department state can trigger initialization:

```
String departmentName =
    student.getDepartment().getName();
```

Hibernate may then execute:

```
SELECT
    id,
    name
FROM department
WHERE id = 10;
```

Lazy loading therefore defers the cost of fetching an association until that association is actually used.

10. Why Lazy Loading Is Useful

Lazy loading is useful because different use cases need different parts of the same entity graph.

For example:

- a student list may need only `id` and `name` ;
- a student details page may also need `department` and `profile` ;
- an administration report may need `department` , `courses` and `projects` .

Using one large eager graph for all three cases wastes database work and memory.

Lazy mapping provides a small default graph. Each use case can then explicitly fetch the associations it needs.

However, lazy loading creates another risk: database queries can be triggered implicitly while Java code navigates relationships. This is how the N+1 problem commonly appears.

11. The N+1 Query Problem

The N+1 problem occurs when:

1. one query loads a collection of parent entities; and
2. additional queries are executed while accessing a relationship for each parent.

Suppose `Student.department` is lazy:

```
@ManyToOne(fetch = FetchType.LAZY)
private Department department;
```

Load all students:

```
List<Student> students =
    studentRepository.findAll();
```

Hibernate first executes one query:

```
SELECT
    id,
    name,
    department_id
FROM student;
```

Now iterate through the results:

```
for (Student student : students) {
    System.out.println(
        student.getDepartment().getName()
    );
}
```

Each department access may initialize a lazy proxy:

```
SELECT * FROM department WHERE id = 1;
SELECT * FROM department WHERE id = 2;
SELECT * FROM department WHERE id = 3;
-- additional queries
```

The total pattern is:

1 query for the students
+
N additional relationship queries

The Java loop looks harmless, but every relationship access may involve a database round trip.

12. First-Level Cache and the Exact Query Count

The term N+1 describes the query pattern. It does not guarantee that exactly N additional SQL statements will always be executed.

Suppose 100 students belong to only three departments.

The initial query loads all students:

1 student query

As departments are accessed, Hibernate may need only:

3 department queries

Once a department has been loaded into the current persistence context, Hibernate can reuse that managed entity for other students referencing the same department.

The result may therefore be:

1 + 3 queries

rather than:

1 + 100 queries

The first-level cache can reduce duplicate primary-key lookups within one persistence context, but it does not solve the underlying fetch-plan problem.

The important issue remains:

Relationship access is causing additional SQL statements that were not visible in the original repository call.

13. N+1 During Collection Iteration

The classic to-many example begins by loading departments:

```
List<Department> departments =  
    departmentRepository.findAll();
```

Suppose `Department.students` is lazy:

```
@OneToMany(  
    mappedBy = "department",  
    fetch = FetchType.LAZY  
)  
private List<Student> students;
```

Now iterate over every collection:

```
for (Department department : departments) {  
  
    for (Student student :  
        department.getStudents()) {  
  
        System.out.println(student.getName());  
    }  
}
```

Hibernate may execute:

```
SELECT * FROM department;
```

```
SELECT * FROM student  
WHERE department_id = 1;
```

```
SELECT * FROM student  
WHERE department_id = 2;
```

```
SELECT * FROM student  
WHERE department_id = 3;
```

There is one query for all departments and one additional query for the student collection of each department.

This is a true N+1 pattern:

```
1 parent query + N collection queries
```

14. Does N+1 Mean Lazy Loading Is Bad?

No.

The N+1 problem can make it appear that lazy loading is bad and eager loading is the solution. That conclusion is too broad.

Lazy loading is useful when an association is not required. The problem arises when the application knows that it will access an association for every result but does not include that association in the query's fetch plan.

The correct principle is:

Keep relationships lazy by default and fetch the required graph explicitly for each use case.

Changing the mapping to `EAGER` is not a reliable N+1 solution.

For example:

```
@ManyToOne(fetch = FetchType.EAGER)
```



```
private Department department;
```

JPA requires the departments to be available eagerly, but it does not require Hibernate to obtain everything using one join query. Depending on the query and provider behavior, Hibernate may still execute secondary selects.

EAGER also applies globally. It loads the association even in use cases that do not need it.

Therefore:

- **LAZY** controls the default timing;
- a query-level fetch plan controls what a specific operation needs;
- neither setting should be chosen without considering the use case and generated SQL.

15. Detecting N+1 with SQL Logging

One of the simplest ways to detect N+1 during development is to inspect the SQL generated by Hibernate.

Enable SQL logging:

```
spring.jpa.show-sql=true  
spring.jpa.properties.hibernate.format_sql=true
```

For parameter values in modern Hibernate versions, logging can also be configured through the application's logging settings:

```
logging.level.org.hibernate.SQL=DEBUG  
logging.level.org.hibernate.orm.jdbc.bind=TRACE
```

Now run the use case and inspect:

- how many SQL queries are executed;
- whether queries repeat inside a loop;
- which Java relationship access triggers each query;

- whether the same association is fetched even when it is not required.

SQL logging is especially useful while learning because it makes the hidden database work visible.

16. Preventing N+1 with an Entity Graph

An entity graph defines which associations should be fetched for a particular operation.

Suppose the default mapping remains lazy:

```
@ManyToOne(fetch = FetchType.LAZY)
private Department department;
```

In Spring Data JPA:

```
public interface StudentRepository
    extends JpaRepository<Student, Long> {

    @EntityGraph(attributePaths = "department")
    List<Student> findAll();
}
```

This method requests `department` as part of its fetch plan.

The important design is:

```
Default entity mapping
Student.department = LAZY

Specific repository method
Fetch Student + Department
```

The mapping remains small for other operations, while this method fetches the association it knows it will need.

Hibernate commonly satisfies this graph using a join, although the exact SQL strategy is controlled by the persistence provider. The entity graph defines what must be fetched, not one universally guaranteed SQL statement.

Fetching Multiple Attributes

Multiple relationships can be included:

```
@EntityGraph(  
    attributePaths = {  
        "department",  
        "profile"  
    }  
)  
List<Student> findAll();
```

Conceptually, this repository method requests:

```
Student  
├─ Department  
└─ Profile
```

Both attributes become part of that method's fetch plan.

Nested Attribute Paths

Spring Data JPA also supports nested paths:

```
@EntityGraph(  
    attributePaths = {  
        "department.manager"  
    }  
)  
List<Student> findAll();
```

This asks for:

Student

└ Department

└ Manager

Nested graphs should be selected carefully. Fetching several large collections in one operation can produce a very large joined result and duplicate rows at the SQL level.
